



软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院



第十二章 微服务架构设计

主要内容

- 1 概述
- 2 架构师的修养
- 3 服务建模
- 4 服务集成
- 5 分解单块系统
- 6 部署



1 概述

- **微服务**是一种分布式系统解决方案，通过使用细粒度服务，使其协同工作，每个服务有其生命周期
- 微服务围绕业务领域建模，可以避免分层架构引发的问题
- 实现了系统本身的设计**支持快速变化**，从而实现快速交付
- 是在领域驱动设计、持续交付、按需虚拟化、基础设施自动化、小型自治团队、大型集群系统等实践逐步流行的背景下，衍生的软件设计变革



1.1 什么是微服务

- **微服务**的定义：协同工作的小而自治的服务

- 特征：

- 小

- 遵循内聚性原则
 - 根据业务的边界来确定服务的边界
 - 服务的粒度能很好地与团队结构相匹配
 - 服务越小，独立性带来的好处越多，然而管理大量服务也会越复杂

能够在两周
内重写一个
服务

- 自治

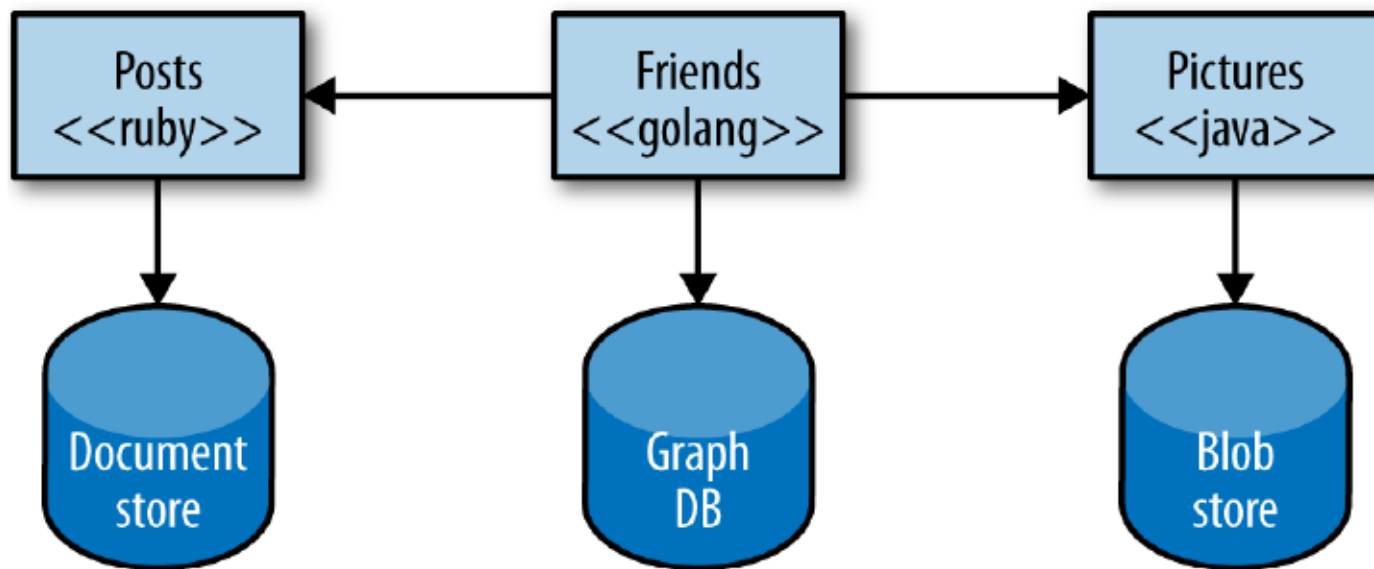
- 每个微服务是独立的
 - 服务间通过网络调用进行通信
 - 服务间相互隔离，避免紧耦合
 - **法则：你是否能修改一个服务并对其进行部署，而不影响其他服务？**



1.2 微服务的益处

● 1 技术异构性

- 可以在不同服务中使用最适合的技术
- 可以使用性能更好的技术栈重构服务
- 支持组织更快地采用新技术
- 社交网站的例子：



1.2 微服务的益处

● 2 弹性

○ 隔板（**bulkhead**）的概念

- 一个服务失效不会导致级联故障
- 服务边界显然是一个隔板

○ 单块系统中服务不可用将导致系统瘫痪，因此采用整体多实例备份

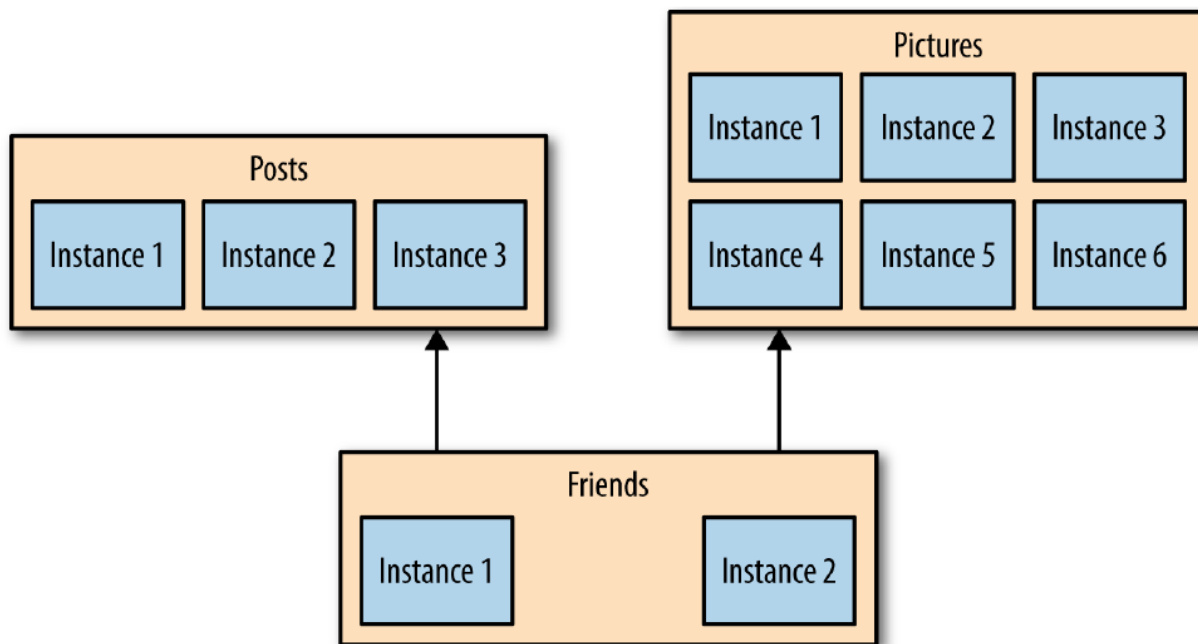
○ 微服务系统能够处理服务不可用和功能降级问题



1.2 微服务的益处

● 3 扩展性

- 若系统中只有一小部分存在性能问题，只要对需要扩展的服务进行扩展即可
- 不需要扩展的服务可以运行在算力要求低的硬件上
- 社交网站的例子：



1.2 微服务的益处

● 4 简化部署

○ 现状：

- 大规模单块应用哪怕只修改一行代码，也需要重部署
- 部署影响大、风险高，因此频率低、每次发布差异大，出错可能性大

○ 微服务：

- 各服务独立部署
- 出了问题只影响一个服务，容易迅速回滚
- 客户可以更快地用到新功能



1.2 微服务的益处

● 5 与组织结构匹配

- 小型代码库使得团队工作更高效
- 易于处理异地团队协作工作
- 服务的所有权易于迁移

● 6 可组合性

- 根据不同目的，以不同方式重用服务
- 应用模式越来越广泛：**Web**、移动原生、移动**Web**、平板、可穿戴、.....
- 微服务提供了积木式连接



1.2 微服务的益处

● 7 对可替换性的优化

- 采用微服务可以轻易重写服务
- 由于微服务的代码库不大，删除它不会有太多感情依赖
- 可替换性提升



1.3 重谈SOA

● SOA的前世今生

- 是一种很好的设计模式
- 服务以独立的形式存在于操作系统的进程中
- 服务间通信通过网络调用，而非进程内调用
- 现状：
 - 通信协议如何选择
 - 第三方中间件如何选择
 - 服务粒度如何确定
 - 不太关心服务划分准则
 - 厂商宣传的成分高，破坏了**SOA**的目标

微服务是
SOA的一种
特定方法



1.4 微服务划分的方案

● 1 共享代码库

- 通过代码库的形式共享功能
- 执行公共任务，一般与业务无关
- 缺陷：
 - 无法选择异构的技术
 - 不具备独立地对系统某一部分进行扩展的能力
 - 每次当库有更新时，需要重新部署整个进程，无法独立地部署变更部分
 - 弹性差
 - 执行公共任务的代码可能成为服务的耦合点



1.4 微服务划分的方案

● 2 模块化

- 模块能对自身进行生命周期管理
- 可被部署到运行的进程中
- 能够在不停止整个进程的前提下对某个模块修改
- 缺陷：
 - 实现模块隔离复杂（如**OSGi**）
 - 模块化语言无法避免类似共享代码库的问题
 - 单进程中的模块隔离在实践中很难做到，往往沦为“共享代码库”
 - **进程边界是实现隔离的最好方法**



1.5 小结

- 采用微服务架构无法避免处理分布式的复杂性
- 部署、测试、监控复杂
- 仍需考虑扩展性、弹性、分布式事务、**CAP**等诸多问题

CAP定理：一个分布式系统中，**Consistency**（一致性）、**Availability**（可用性）、**Partition tolerance**（分区容错性）最多只能同时保证两个，三者不可兼得。

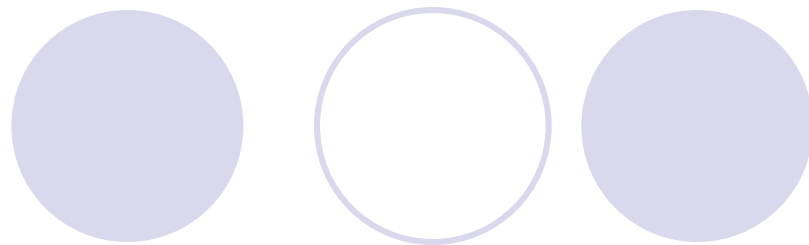


2 架构师的修养

- 对软件架构师的思考：
- 架构师的职责不仅要使用户满意，而且是保证构建的系统适合开发人员在其上工作（设计、编码、测试、运维等）
- 架构师专注大方向、偶尔会关注到细节
- 不但满足当前需求，还能够应对将来的变化
- 架构师的能力要求：



架构师的能力要求



● 1 分区

○ 关注服务如何划分

- 服务边界、粗粒度的服务群组分区
- 服务间如何交互
- 系统整体的健康状态

○ 服务自治

- 每个服务内部允许团队采用任意技术栈或数据存储技术
- 掌握“度”



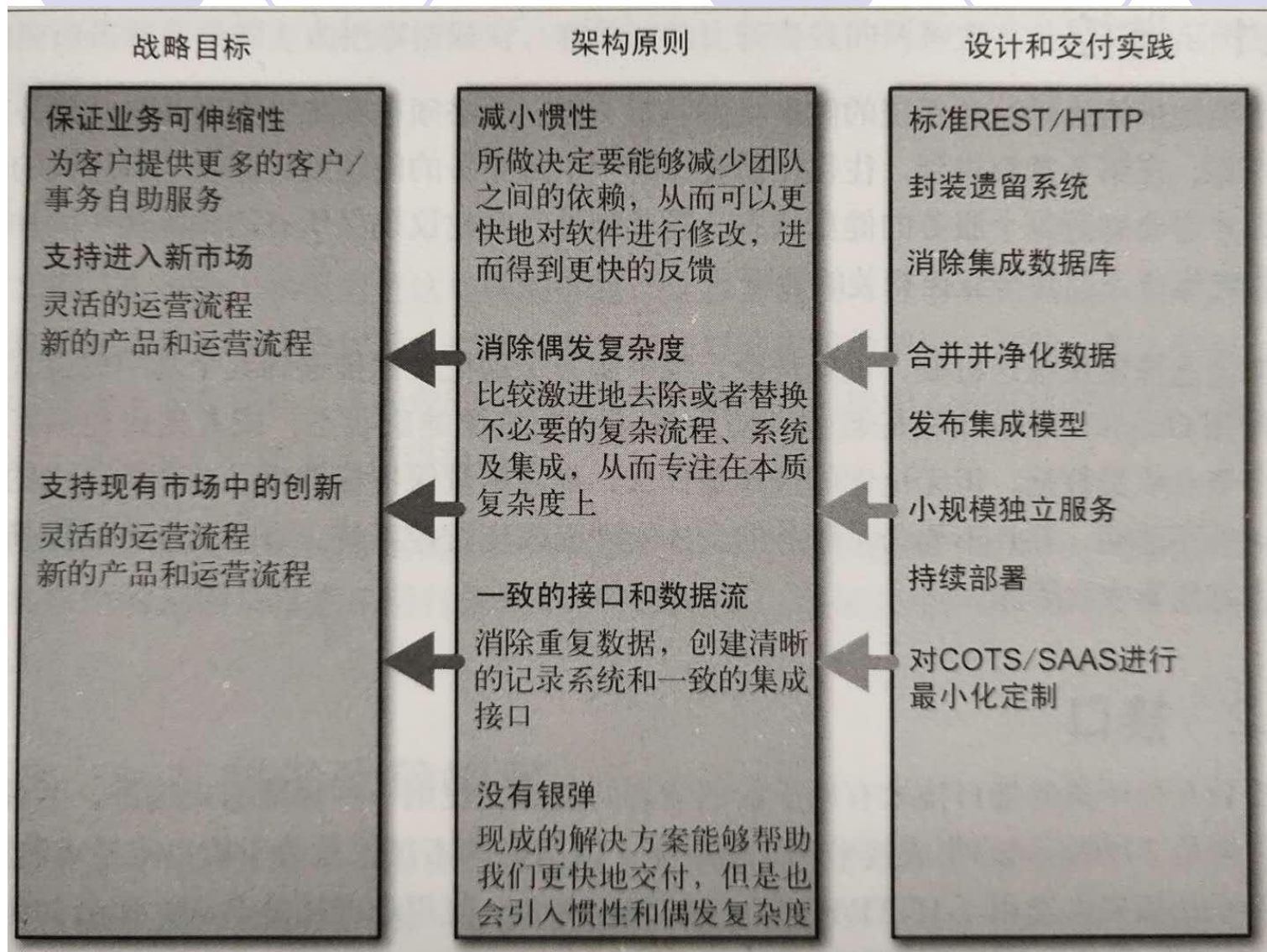
架构师的能力要求

● 2 制定技术原则

- 技术选型与组织战略目标一致
- 找出符合战略目标的技术原则和约束
 - 原则不多于**10**个
 - 约束一般不能改变
- 通过技术实践保障和巩固原则的实施，如
 - 代码规范
 - 集中处理日志数据
 - **HTTP/REST**作为标准集成风格
 -
- 注意约束，如
 - 团队使用**Java**技术栈等



技术原则与实践



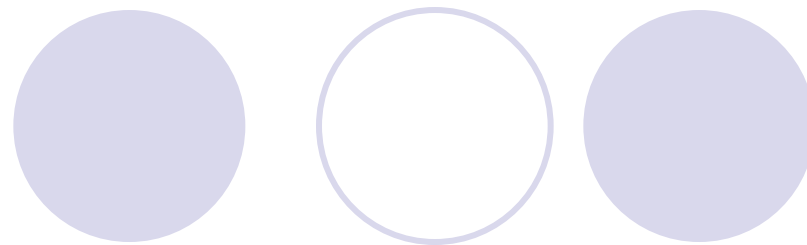
架构师的能力要求

● 3 给出衡量标准

- 能够清晰地描述出跨服务系统的健康状态
 - 确保所有服务采用同样的方式报告健康状态和与监控相关的数据
 - 如：状态信息推送；轮询收集数据；日志等
- 标准接口
- 服务调用的异常处理方法



架构师的能力要求



● 4 代码治理

○ 提供范例

- 范例是真实可运行的代码

○ 给出服务代码模板

- 支持对代码模板裁剪
- 不强制使用庞大的框架
- 框架应能简化开发
- 注意重用代码所引入的服务间的耦合



架构师的能力要求

● 5 处理技术债务

- 由变更引起的技术偏离——“技术债务”问题
 - 权衡是否需要偏离技术愿景
 - 决定何时偿还技术债务

● 6 监督执行

- 承担技术治理的职责，保障符合技术愿景并演化
- 与实现团队一起工作
- 可成立治理小组

● 7 团队建设

- 帮助技术团队成长



3 服务建模

- 如何确定服务间的边界？
- 最大化微服务的优点，避开缺点



3.1 什么服务是好的

● 松耦合

- 能够独立修改和部署单个服务而不需要修改系统其他部分
- 松耦合的服务应该尽可能少地知道与之协作的那些服务的信息
- 避免过度通信

● 高内聚

- 相关的行为聚集在一起
- 不相关的行为分开放置
- 找到问题域的边界



3.2 业务边界

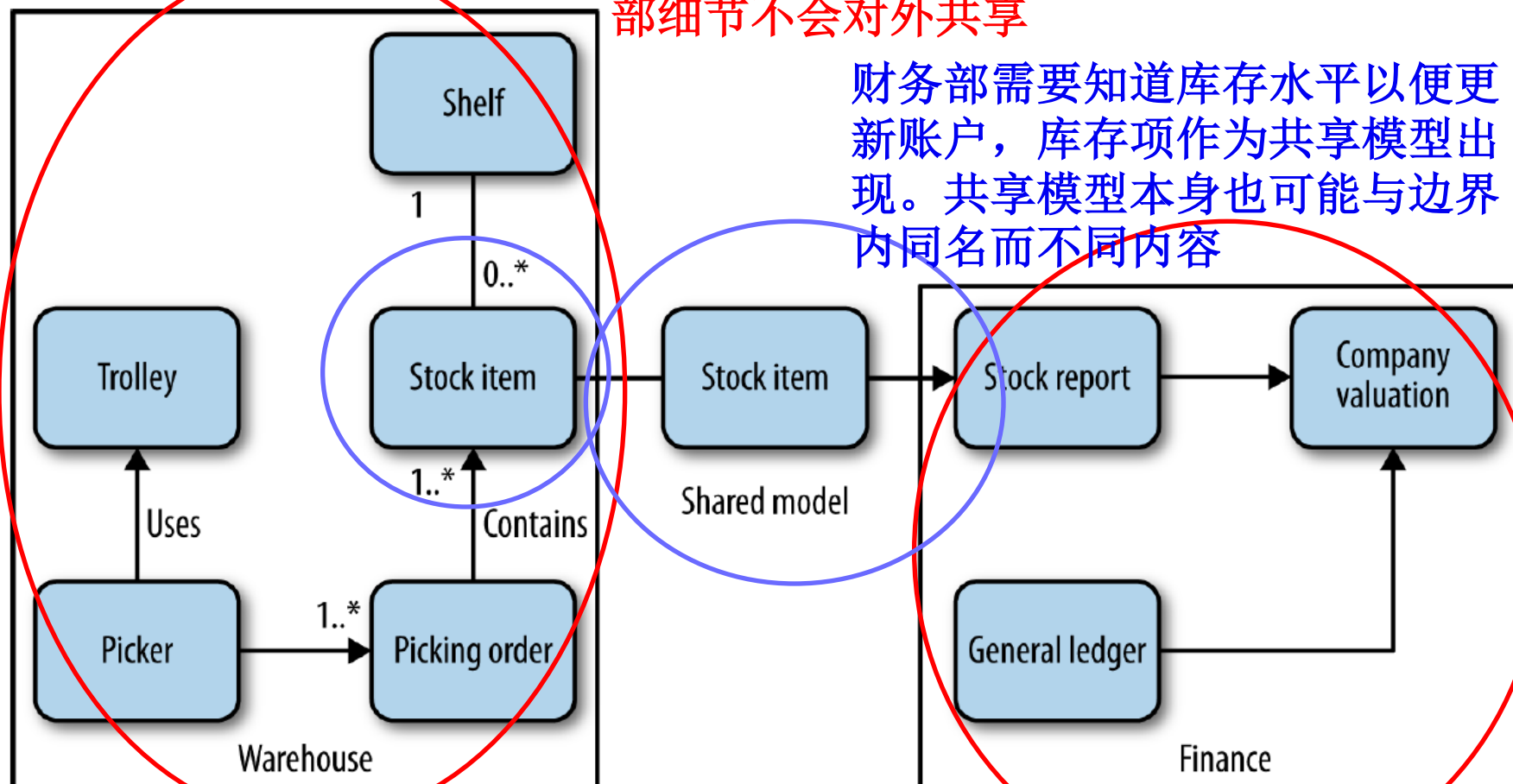
- 边界上下文 (**bounded context**)
 - 由显式边界限定的特定功能
- 共享模型
 - 对不同边界间的共享物建模
- 类比：
 - 细胞膜的例子
 - 隔离了细胞内外
 - 确定了什么物质可以通过细胞膜
- 例子：音乐商店(**MusicCorp**)的财务部和仓库



共享模型

仓库和财务部包含的内部细节不会对外共享

财务部需要知道库存水平以便更新账户，库存项作为共享模型出现。共享模型本身也可能与边界内同名而不同内容



应共享模型而不是共享内部表示



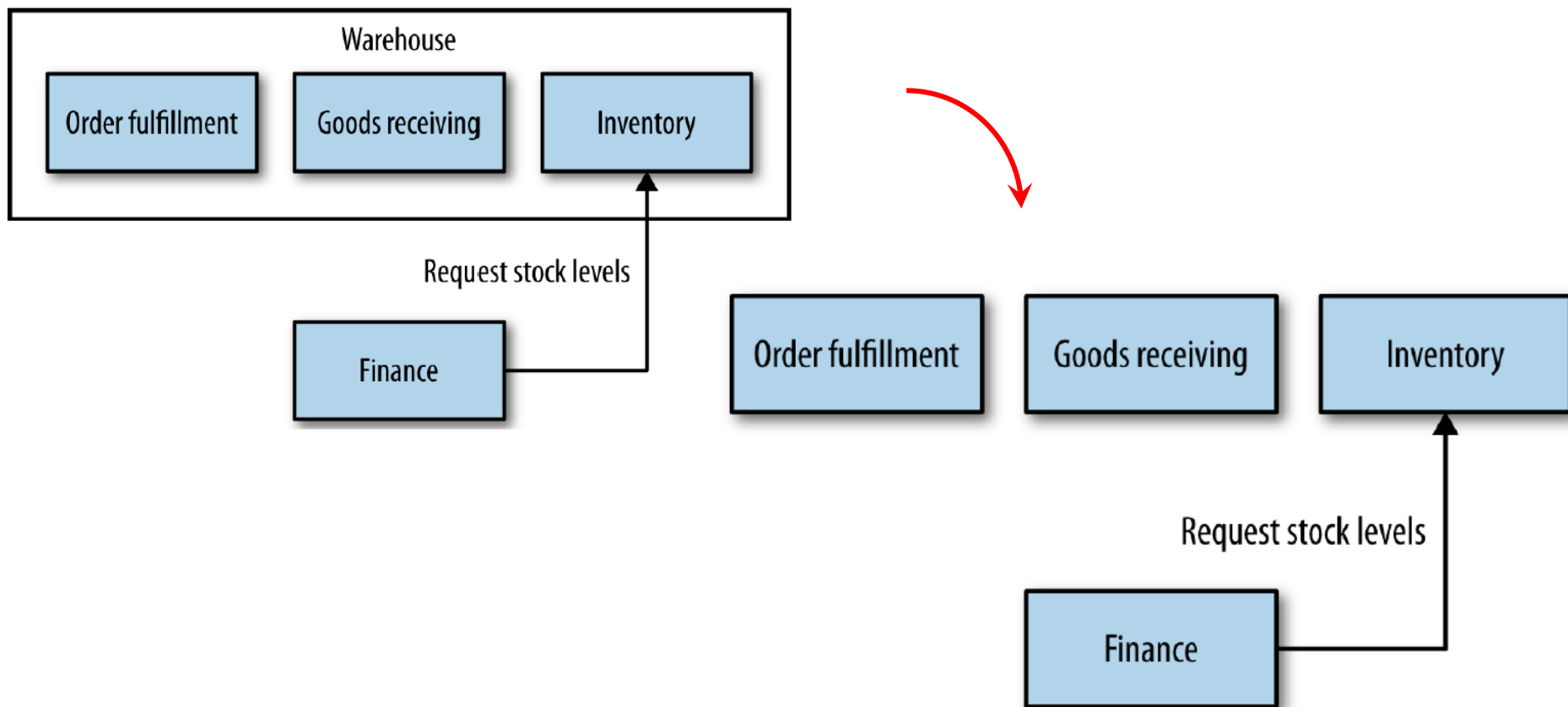
3.2 业务边界

- 微服务应清晰地与边界上下文保持一致
- 可以先用模块划分在同一进程中运行，待稳定后再作为微服务进行划分
 - 单块系统→微服务架构
- 过早将一个系统划分成微服务有可能代价很高，尤其是新的业务领域
- 将一个已有的代码库划分为微服务，要比从头开始构建微服务容易很多
- 应从功能而不是共享数据角度来考虑边界上下文



3.3 业务边界的层次划分

● 边界上下文的级别



3.3 业务边界的层次划分

- 划分准则：
 - 根据组织结构来划分
 - 根据测试要求而定



3.4 对服务划分的一些其他考虑

- 相同领域业务概念尽量封装到边界内
- 谨慎使用技术边界（分层架构对微服务可能并不合适）
- 按地理位置划分
- 按组织机构划分



4 服务集成

- 集成是保持服务自治的关键
 - 好的集成技术：能独立地修改和发布服务
 - 不好的：通信效率低下，为整个系统带来灾难



4.1 理想的集成技术

- 避免破坏性修改

- 对某个服务的修改不应影响其消费者

- 保证**API**的技术无关性

- 微服务之间通信方式与技术无关至关重要

- 使服务易于消费方使用

- 消费方应该可以使用任何技术来实现
- 提供例子客户端代码能够帮助消费方使用

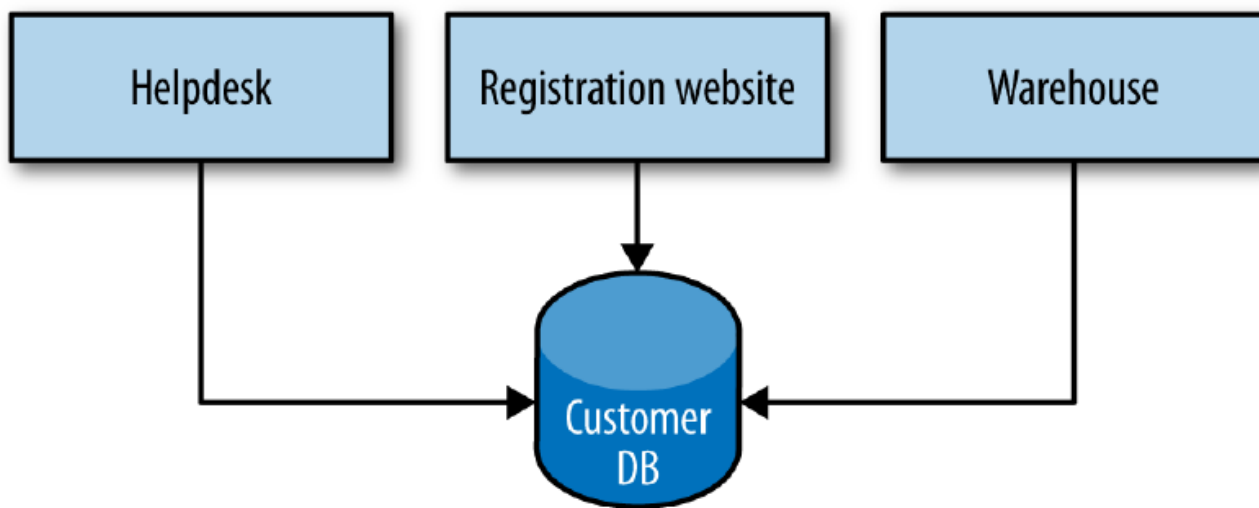
- 隐藏内部细节

- 不应采用所有倾向于暴露内部实现细节的技术



4.2 共享数据库

- 是最常见的集成方法
 - 通过数据库实现两个服务间的信息交互
 - 例如：



4.2 共享数据库

● 缺陷：

- 使用它的服务都能够查看内部实现细节，并与其绑定在一起
- 数据库结构的修改会影响所有服务
- 对数据库中实体的操作行为放在何处？
 - 放在使用它的服务中，这样对数据库进行操作的相似逻辑会出现在很多服务中

违背了低耦合
和高内聚的原
则！



4.3 服务集成模式

● 1. 同步与异步通信模式

- 采用 **请求/相应**和 **基于事件**的协作风格

- **请求/相应**

- 同步：自然的方式

- 异步：发起一个请求，然后注册一个回调，当服务端操作结束后，会调用该回调

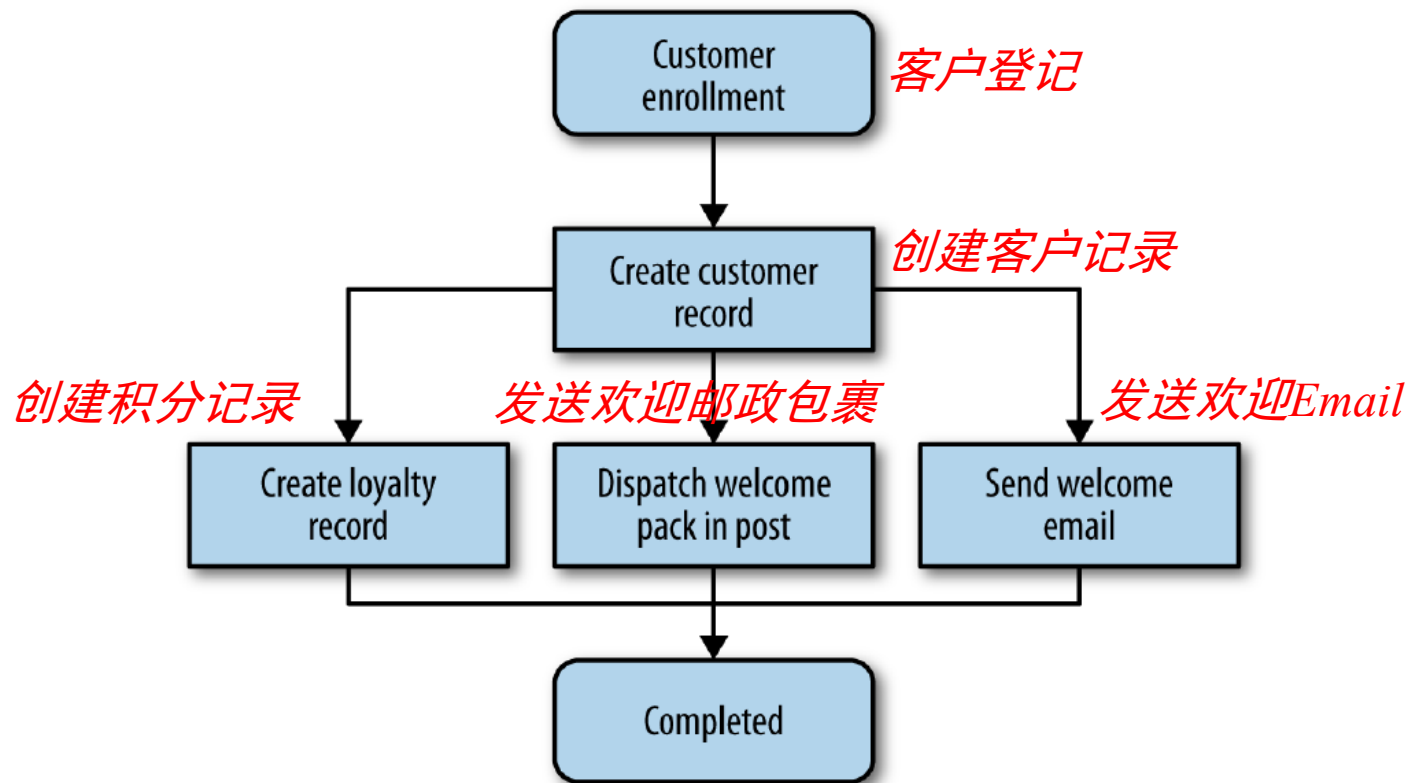
- **基于事件**

- 天生异步：客户端不发起请求，而是发布一个事件，等待其他协作者接收该消息，此协作者自己知道该怎么做



4.3 服务集成模式

● 2. 跨服务边界的业务流程

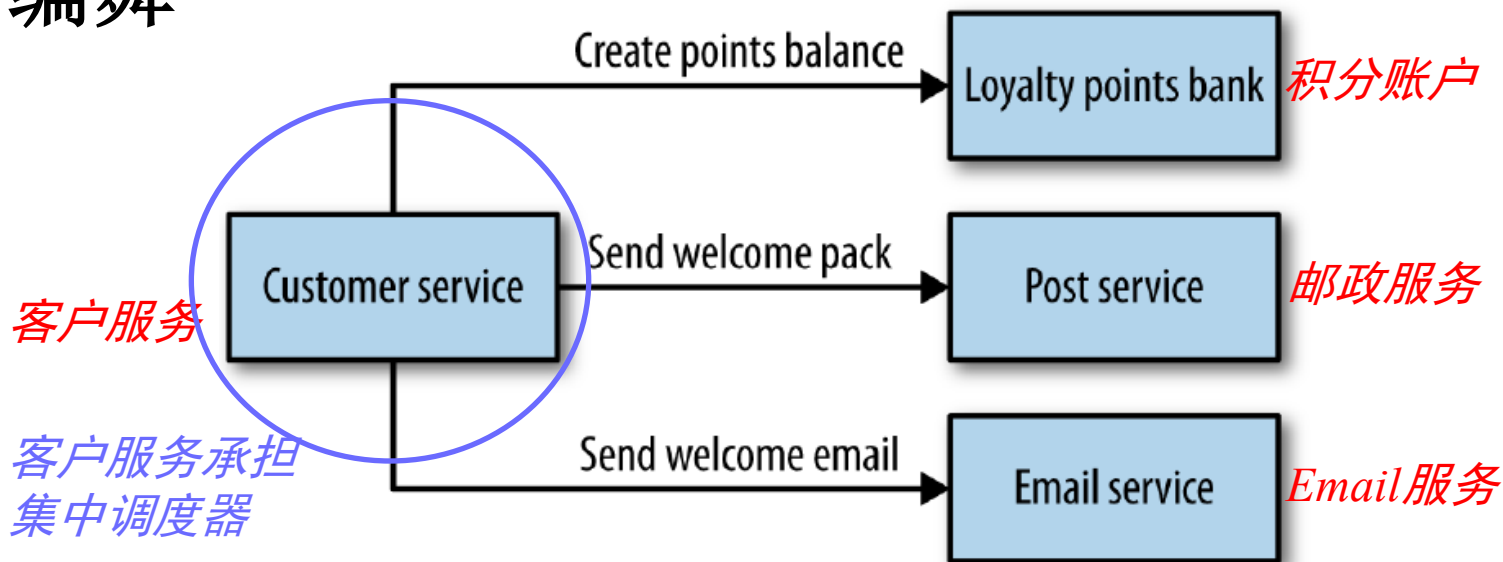


○ 两种服务协作模式



两种服务协作模式

● 编舞



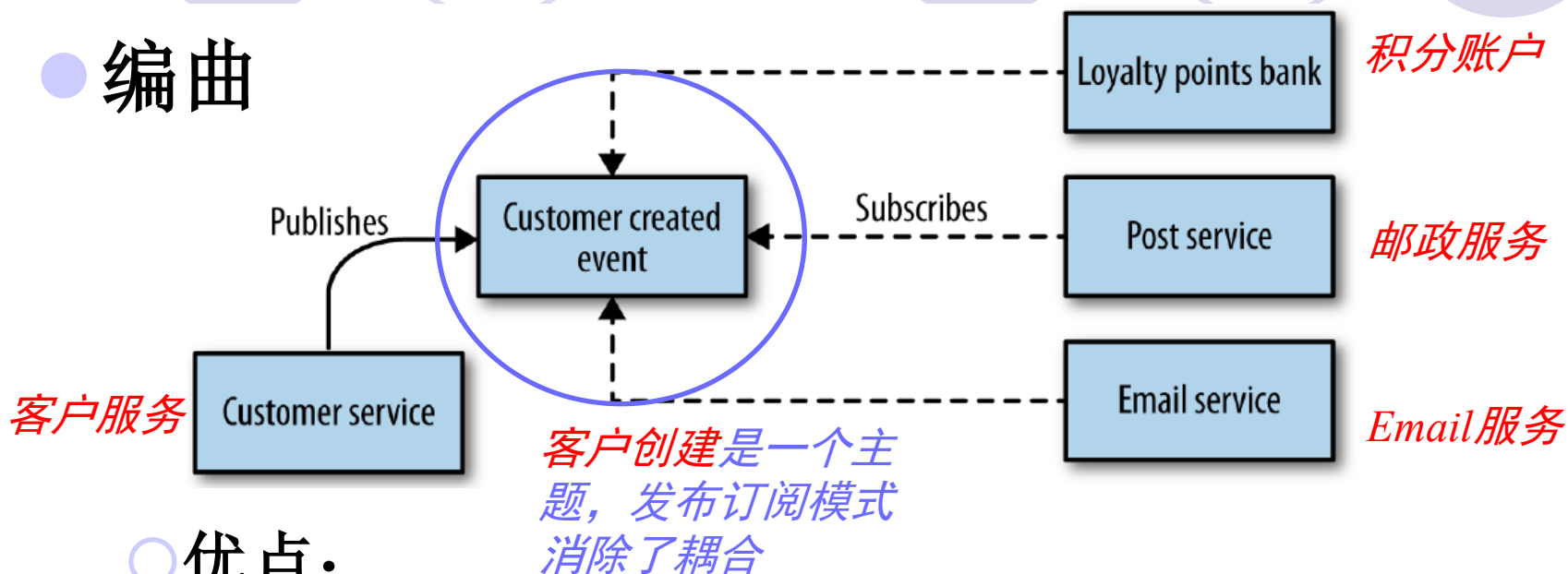
○ 缺陷:

- 中心调度器承担太多的职责，是 **中心枢纽** 和很多业务处理逻辑的起点，其可靠性和性能至关重要
- 与其打交道的其他服务沦为 **傻服务**，甚至只会 **CRUD**



两种服务协作模式

● 编曲



○ 优点：

- **低耦合**，能更灵活地对现有系统进行变更
- 每个服务足够**聪明**，能够很好地完成自己的任务

○ 是缺点吗？

- 可能需求额外工作来对业务流程做跨服务监控，保证每个服务都能执行成功



4.3 服务集成模式

● 3. 服务调用方法

○ 同步调用：

○ 1) **RPC**远程过程调用

- 客户端和服务端依赖于接口，可以采用不同技术栈
 - 如**Web Services**、**Thrift**、**protocol buffers**
- 客户端和服务端采用相同技术栈
 - 如**Java RMI**



4.3 服务集成模式

● 3. 服务调用方法

○ 1) RPC远程过程调用

● RPC优缺点：

- 使程序员能够使用普通方法调用而忽略其他网络通信细节
- 技术耦合性一般较高，建议选择仅依赖接口的技术
- 受网络质量影响大
- 客户端和服务端耦合性高
 - 部署无法分离
 - 无法独立变更

● 最佳实践：

- 不要对远程调用过度抽象，以至于网络因素被隐藏起来
- 确保可以独立地升级服务端的接口而不用强迫客户端升级
- 在客户端中不要隐藏进行网络调用的事实



4.3 服务集成模式

● 3. 服务调用方法

○ 2) REST

- **Representational State Transfer**: 表现层状态转化
- 资源在网络中以某种表现形式进行状态转变
 - 资源，即数据，由**URI**来指定
 - **Representational**: 资源的某种表现形式，比如用**JSON**, **XML**, **JPEG**等，在**HTTP**请求的头信息中用**Accept**和**Content-Type**字段指定
 - **State Transfer**: 状态变化，通过**HTTP**动词实现。即通过**HTTP**动词操作资源的表现形式实现对资源的操作，包括：**get**, **post**, **put**, **delete**等
 - 这时候**HTTP**头和有效载荷都包含业务逻辑，例如**HTTP**方法对应**CRUD**操作，**HTTP**状态码对应操作结果的状态



4.3 服务集成模式

● 3. 服务调用方法

○ 2) REST

● 优缺点：

- 将**HTTP**作为应用协议，而不是传输协议，更适合**Web**
- 把软件工程师从“操作”的泥淖中抽出来，使其更关注于“资源”，更进一步说是“数据模型”。而关注“数据模型”后的好处在于，能够帮助不同组件之间更好的解耦，消除依赖关系，以及增大系统拓展性
- 是专门为交互的性能和可伸缩性进行过优化的一种架构风格
- **HTTP**是一个同步的请求响应式协议，不能天然地支持服务端发起的通知（点对点），但这又是经常需要的
- **HTTP**的动词太少，在处理复杂业务时能力不足
- **HTTP**对低延迟通信而言不是最好的选择，建议使用基于**TCP**的技术，如**WebSocket**



4.3 服务集成模式

● 3. 服务调用方法

- 异步调用：

- 1) 消息代理（消息中间件）

 - 如，**RabbitMQ, Kafka, RocketMQ**等

- 2) 使用**HTTP**来传播事件

 - 如，符合**REST**的**ATOM**规范



4.3 服务集成模式

● 4. 微服务架构中的代码共享

- 避免重复代码一般是合理的
- 在微服务间应避免使用共享库
 - 共享库会导致服务与消费者间的过度耦合
 - 可以将共享代码复制到每个服务中，以防止耦合发生
- 经验：在微服务内部避免重复代码是正确的，但在服务之间可以适当违反此原则
- 慎用客户端库



4.3 服务集成模式

● 4. 微服务架构中的代码共享

○ 使用服务的 **客户端库**的注意事项

● 优点：

- 简化对服务的使用
- 避免不同消费者之间存在重复的与服务交互的代码

● 缺点：

- 服务端的逻辑会潜入客户端库，造成耦合
- 修复一个服务端问题的同时，需对所有客户端进行修改
- 限制了服务消费者对技术的选择

- 最佳实践：保证客户端库中仅包含处理底层传输协议的代码，如服务发现、故障处理等



4.4 版本管理

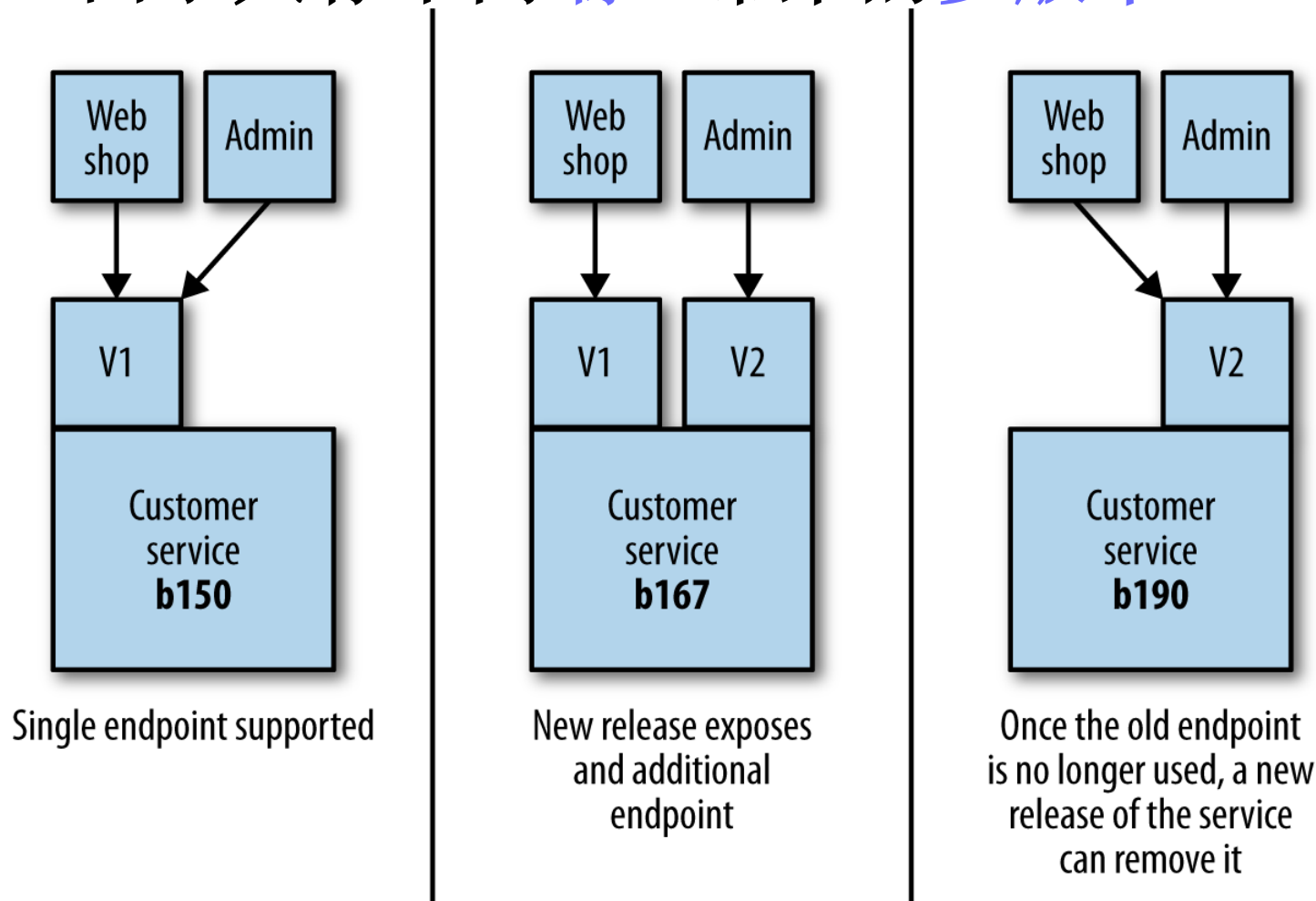
端口的
多版本

- 1 由于共存不同端口带来的多版本
 - 端口(Endpoints)可以使微服务独立于彼此进行发布
 - 有时无法避免破坏性修改, 会影响端口, 此时可以部署一个同时包含新老端口的版本
 - 一旦所有服务消费者不再访问老端口, 就可以删除它及其相关代码
 - 可以将老版本的实现转换到新版本上
 - 例如, 对V1端口的调用转换为对V2的调用
 - 这样以后删除老版本代码比较容易操作



4.4 版本管理

1 由于共存不同端口带来的多版本



4.4 版本管理

服务的
多版本

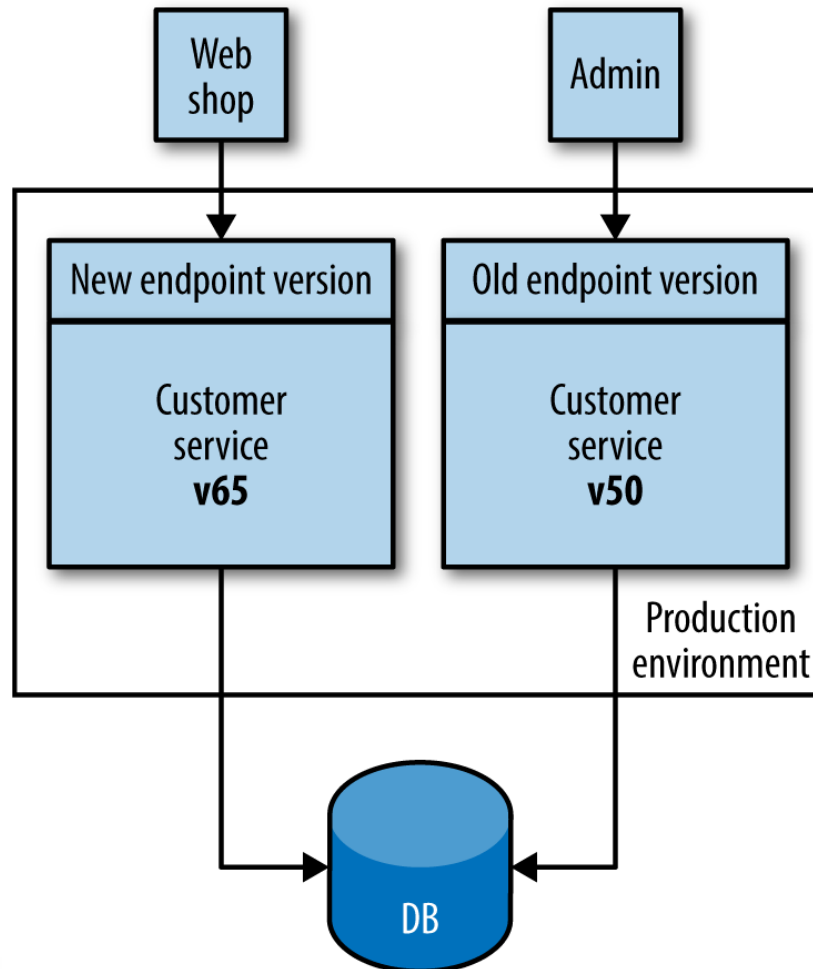
● 2 使用同一个服务的多个版本

- 系统中同时运行不同版本的服务，有些用户（老用户）用老服务，新用户用新服务
- 除非迫不得已，例如：
 - 用户拒绝升级
 - 用户遗留设备与老版本**API**绑死
- 否则不建议大量使用，原因：
 - 服务内有**Bug**，每个版本都需要修改，并分别部署
 - 维护代码分支，引入复杂问题
 - 服务路由比较复杂
- 建议：同一微服务中暴露两套**API**



4.4 版本管理

- 2 使用同一个服务的多个版本
- 例如



4.5 用户界面

● 1 微服务架构中的UI

- 不应对待网页端和移动端区别对待
- 很难预测用户会怎样使用提供的**API**来开发**UI**，所以最好将**API**设计得比较细粒度，比如通过微服务暴露**API**
- 把微服务进行组合，为**PC版Web**程序、移动设备、可穿戴设备等提供不同的体验



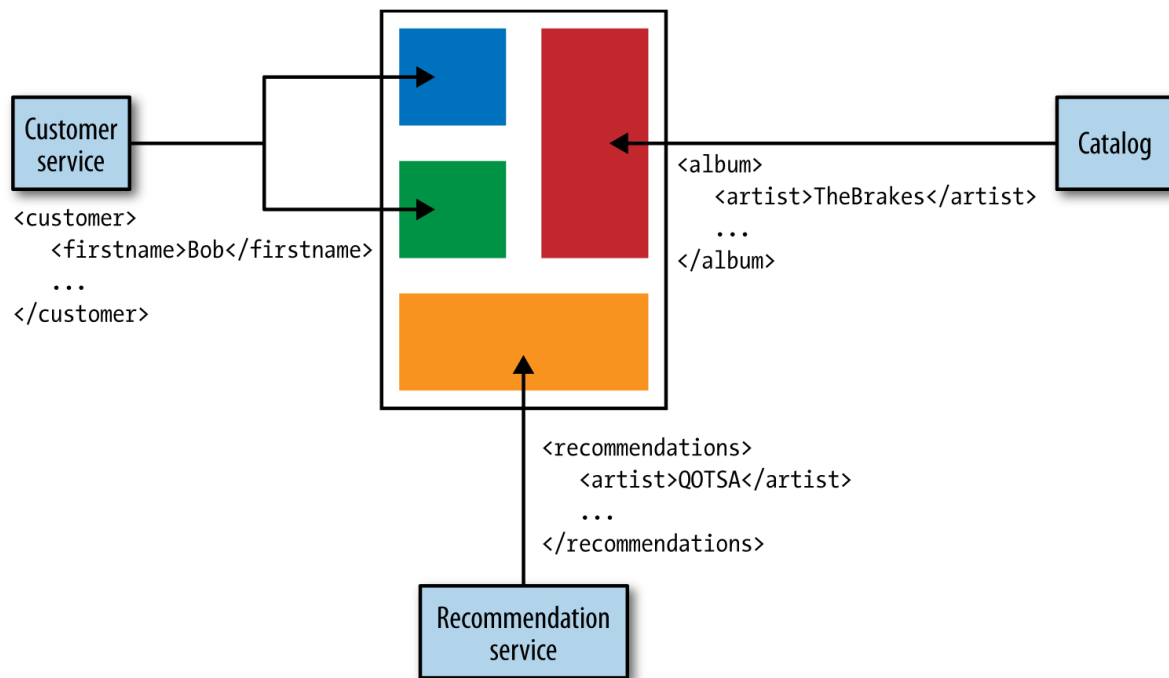
● 2 约束条件

- 考虑不同交互方式中存在的约束
 - PC的分辨率、浏览器内核的限制
 - 移动设备对分辨率、网络条件、耗电情况的约束
- 不同设备操作方式的不同
 - PC上可用鼠标精确点击
 - 移动设备上单手操作的便利性
- 相同的核心服务，需要应对不同的应用场景约束

4.5 用户界面

● 3 几种创建UI的方法

○ 1) 直接调用服务API来创建UI



缺点

- 很难为不同的设备定制不同的响应
- 考虑团队分工，UI由另外的团队开发？退化为分层模式



西北工业大学

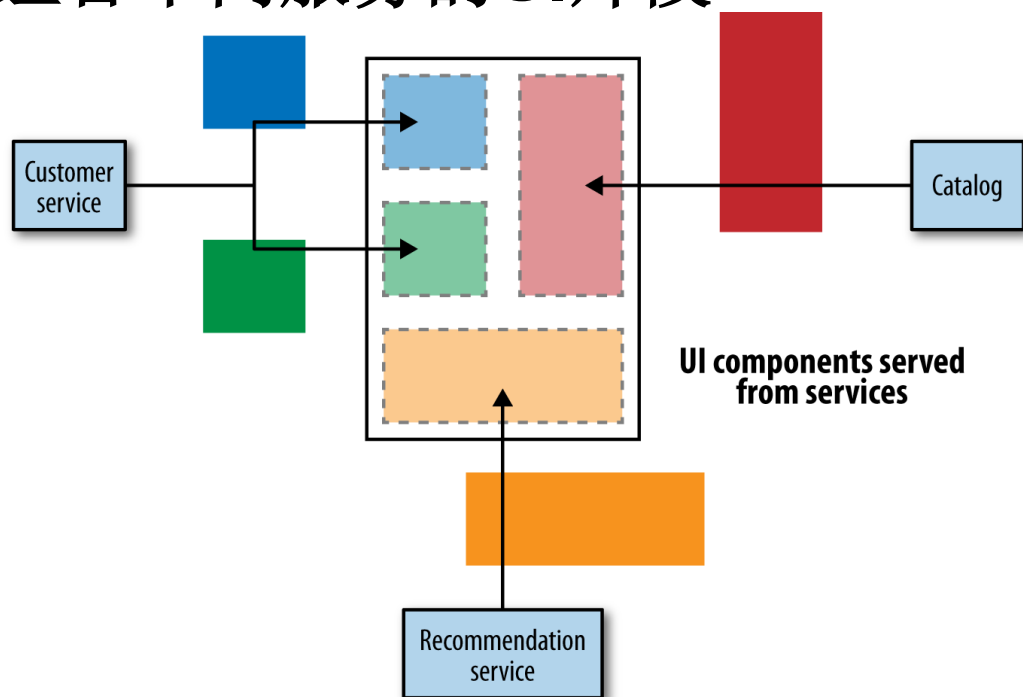
NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.

4.5 用户界面

● 3 几种创建UI的方法

○ 2) 组合不同服务的UI片段



- 与构建微服务的团队对服务的所有权匹配得较好
- 需保证UI用户体验的一致性，并视服务是否适合提供UI

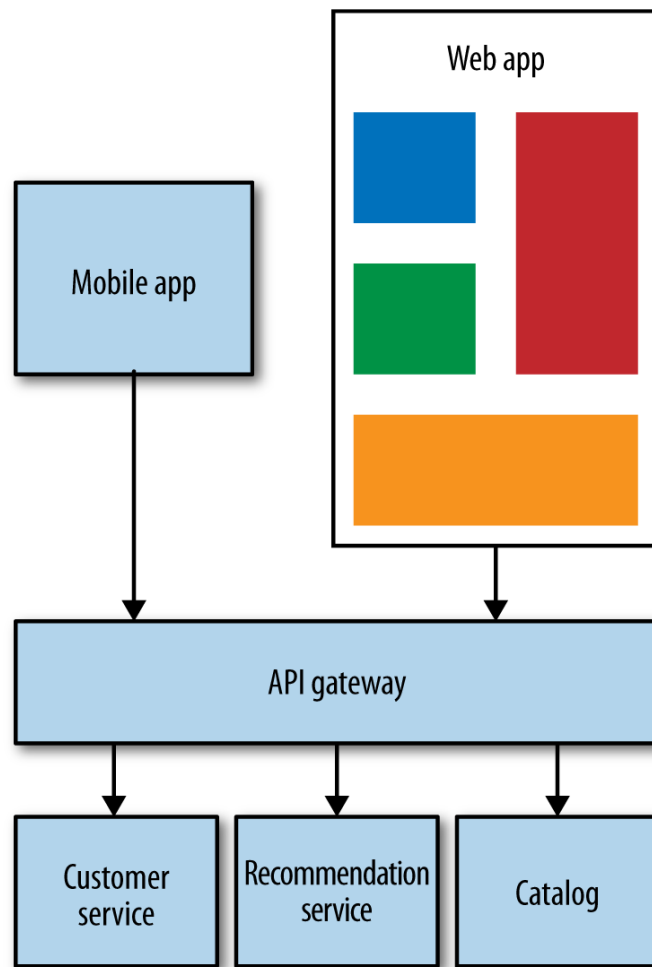


4.5 用户界面

● 3 几种创建UI的方法

○ 3) 给不同设备提供不同UI

- 使用服务端的聚合接口或API入口
- API入口可以对多个后端调用进行编排，为不同的设备提供定制化内容
- 缺陷：
 - API入口容易变厚，包含的逻辑过多，难以维护
 - 失去了不同UI之间的隔离性，限制了彼此独立的发布能力

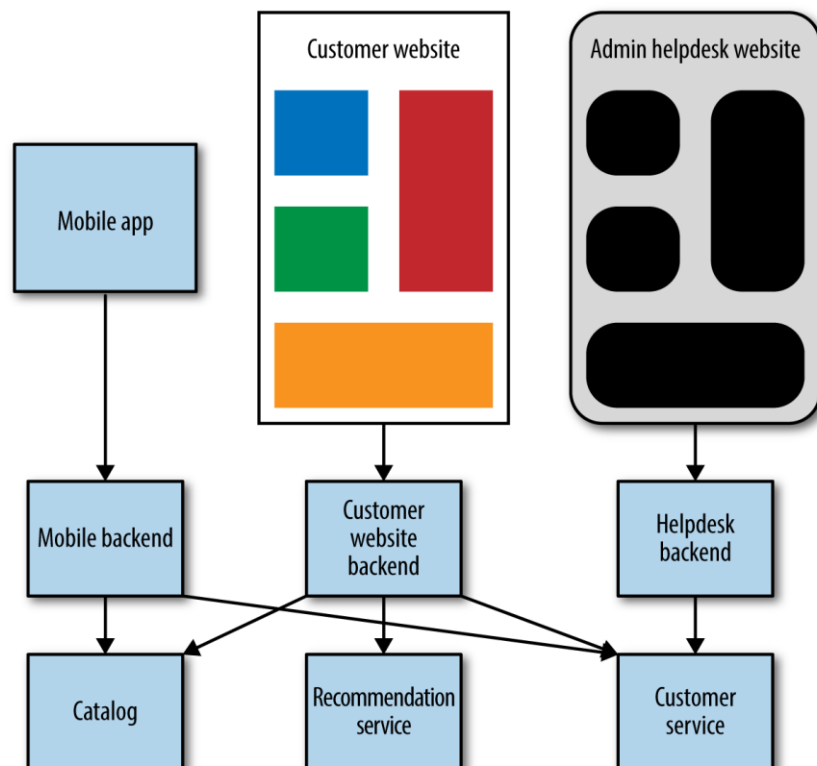


4.5 用户界面

● 3 几种创建UI的方法

○ 4) 为前端提供 **后端**

- 后端是UI的一部分，嵌入服务端
- 注意：将业务逻辑放在服务中，不要泄露到后端，后端应该仅包含与实现某种特定的用户体验相关的逻辑



5 分解单块应用

- 单块应用不符合高内聚、低耦合的原则
- 识别 **接缝**，从 **接缝** 处抽取出相对独立的一部分代码
- **接缝** 很可能成为服务的边界
- 边界上下文 一般可以被看成接缝
- 代码中的边界可以尝试从编程语言的命名空间来找，例如Java中的**package**



5.1 分解单块应用时的考虑因素

- 重构单块应用需循序渐进
- 判断哪些边界上下文中的代码可能会有大量修改，从这个边界上下文开始，使其成为自治单元
- 使位于不同地理位置的团队承担相对独立的工作
- 将需要独立地做监控、传输数据、数据保护和安全防护的部分分离为独立的服务
- 特定技术强相关的代码分离到独立的服务中



5.1 分解单块应用时的考虑因素

- 例：音乐商店(**MusicCorp**)包含边界上下文

- 产品目录

- 财务

- 仓库

- 推荐

- 考虑因素：

- 库存管理需要大量修改

- 产品目录是异地团队开发

- 财务需要安全防护

- 开发了新的推荐算法



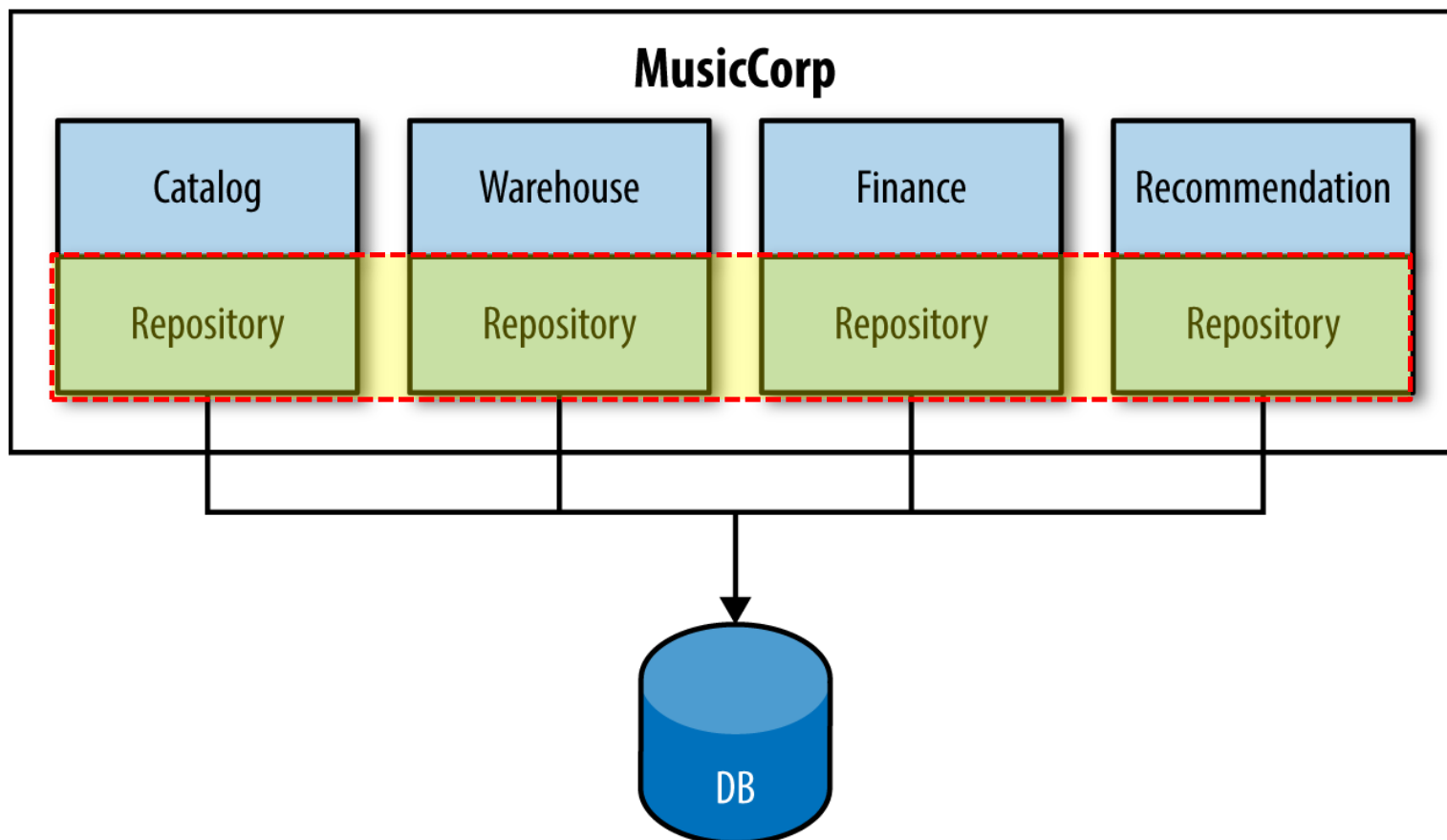
5.2 分解数据库

- 原因：服务集成时尽量不使用共享数据库
- 方法：对数据库访问代码进行分离
 - 按照边界上下文对数据访问层进行分解
 - 需要注意分解后表的外键关联
 - 使用可视化工具，如**SchemaSpy**可以展现表关系
 - 视不同的情况处理：
 - 跨不同上下文的表之间的耦合（关联）
 - 同一张表被多个边界上下文使用



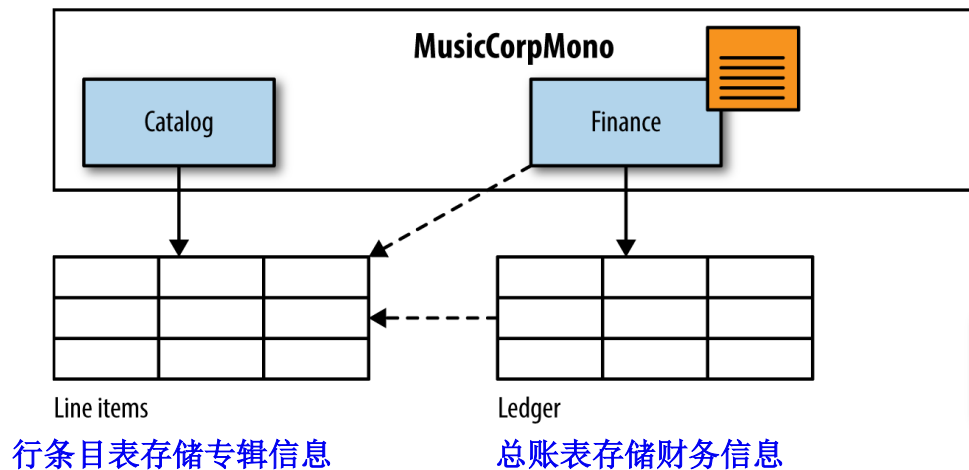
5.2 分解数据库

● 例子: MusicCorp

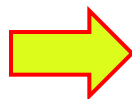


5.2 分解数据库

1 打破外键关联

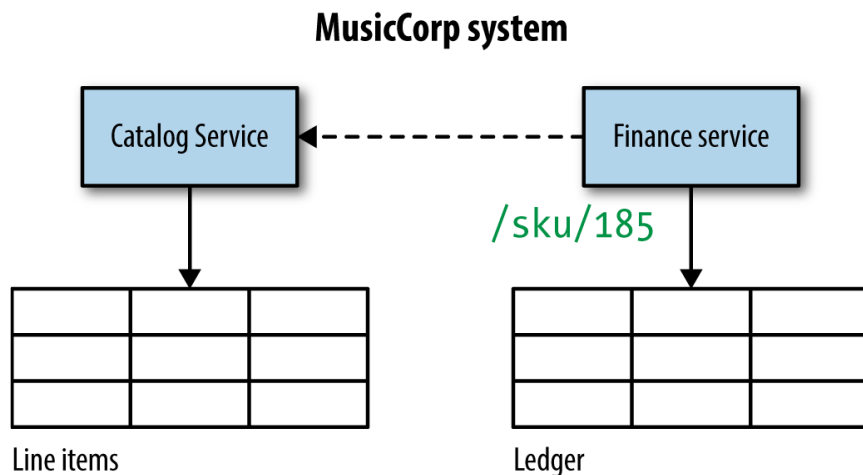


月报表需要“卖了**400**份张学友的饿狼传说，挣了**5000**元钱”的信息，此信息来自两张表



问题：

1. 性能问题？测试后可以接受就大胆去做
2. 外键关联？放到代码中来实现表间约束，如跨服务的一致性检查、周期性触发清理数据等。有时，数据是否一定保持一致由业务决定，如删除产品，相关订单如何处理



- 让财务部分的代码通过产品目录服务暴露的**API**来访问数据，而不是直接访问行条目表
- 需要做两次数据库调用来生成报表



5.2 分解数据库

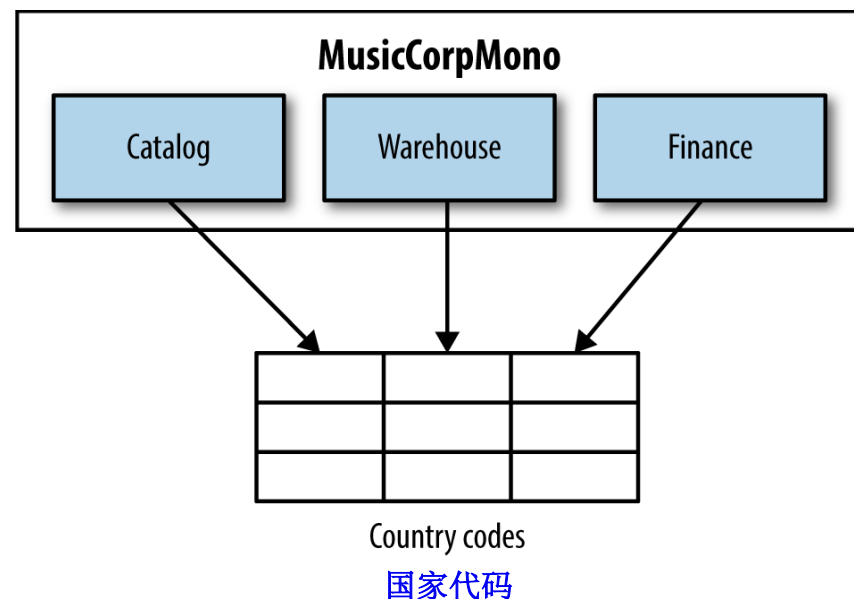
● 2 共享静态数据

○ 分解方案

- 为每个服务复制一份
- 放入代码或配置文件
- 放入一个单独的服务中

○ 权衡

- 方案一、二都会有潜在的一致性问題，但是方案二比修改数据库简单得多
- 方案三需要考察数据量、数据操作规则复杂性是否值得
- 一般方案二能较好地解决问题



5.2 分解数据库

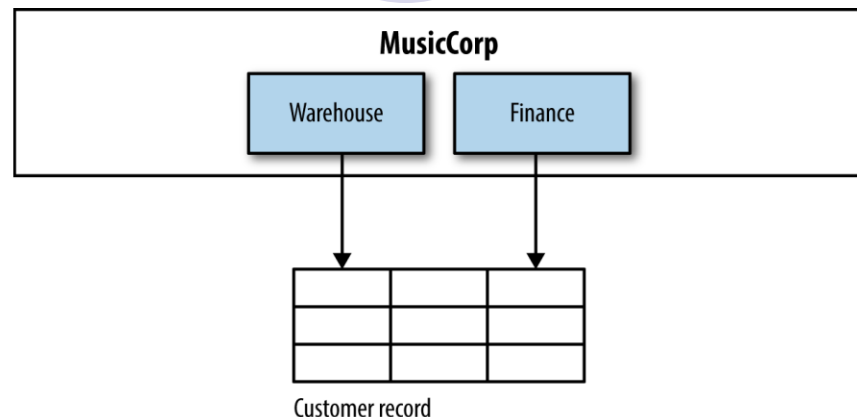
● 3 共享可变数据

○ 问题描述

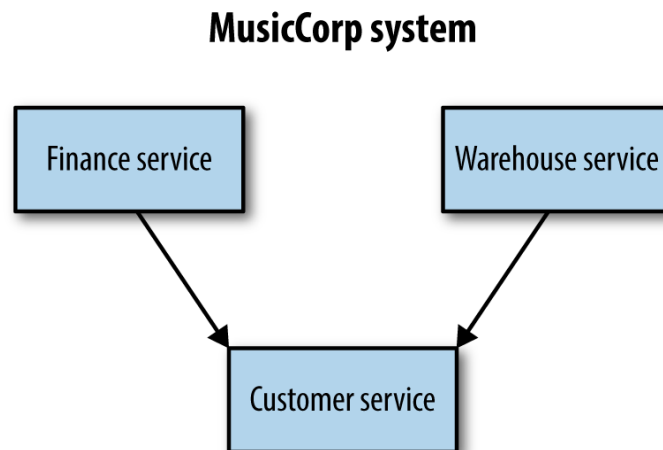
- 两个边界上下文都会向同一个表中写入或读取数据

○ 方案

- 创建一个新服务(客户服务), 让财务和仓库服务通过**API**来访问此服务
- 丧失的性能经测试若能接受, 优先保证服务划分清晰



财务会追踪客户产生的订单、退款等信息, 仓库也会在客户订单被分发或接受之后更新订单信息



5.2 分解数据库

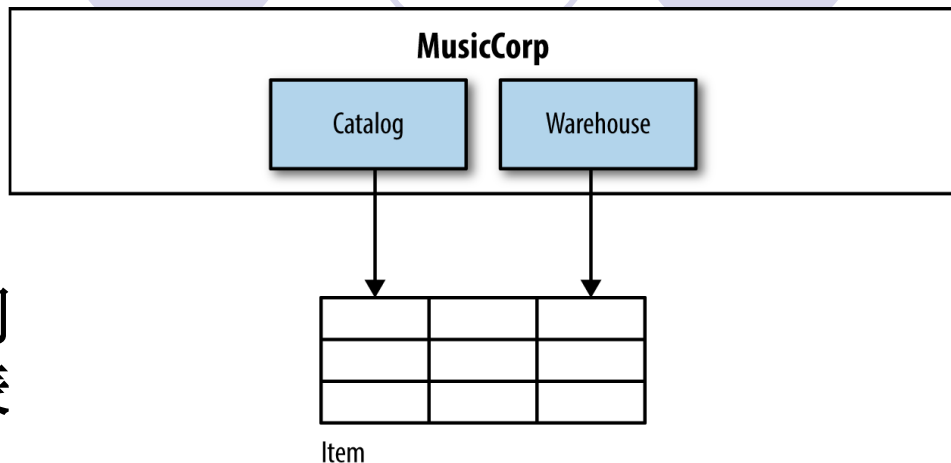
● 4 共享表

○ 问题描述

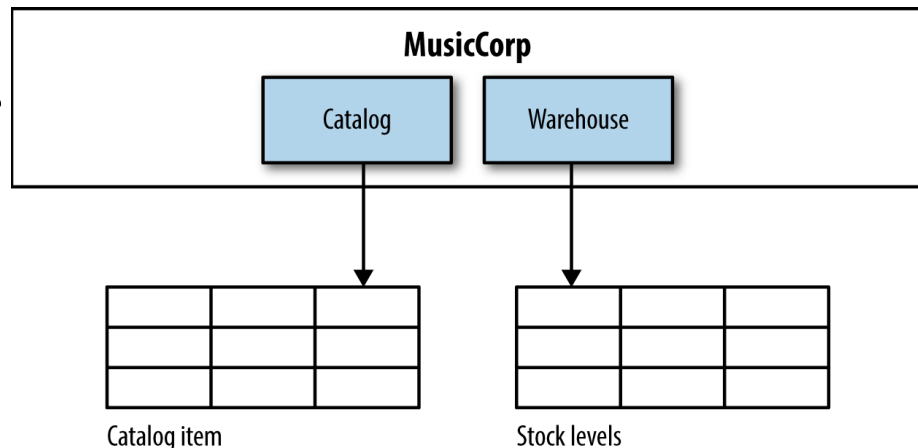
- 两个边界上下文将不同的关注点放在了一张表中

○ 方案

- 遵循分离关注点原则，对仓库创建库存水平表、对产品目录详情创建目录项表

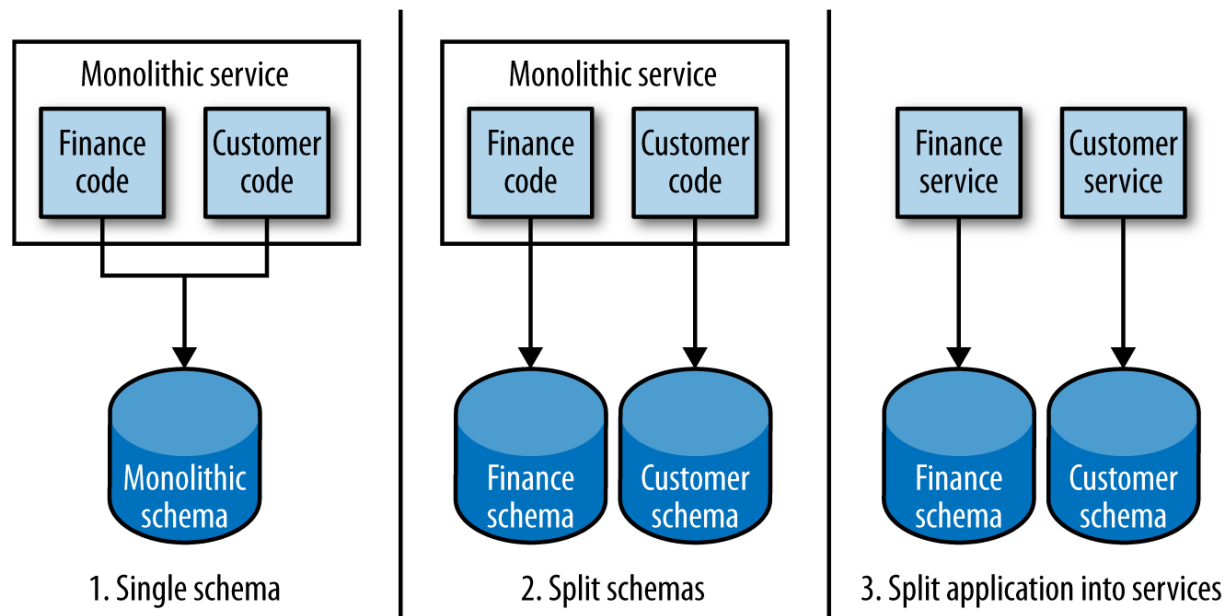


产品目录需要存储产品条目的名字和价格，仓库需要保存仓储的电子记录，放到了同一张**Item**表中



5.3 分解单块应用的方法

- 通过重构数据库来分解单块应用



- 先分表，满意后再分服务

- 分表后对某个动作而言，可能对**DB**的访问变多，也可能会破坏事务的完整性，会对应用程序造成较大影响，可能需要随时回退。待满意后，再分服务



6 部署

- 部署单块应用很简单，但是微服务依赖非常复杂，需要深入研究其部署方法
- 持续集成方法（**CI, Continuous Integration**）

- 是否每天检入代码到主线？
- 是否有一组测试来验证修改？
- 当构建失败，团队是否把修复CI当成第一优先级的事情来做？

CI是一种软件开发实践，即团队开发成员经常集成他们的工作，通常每个成员每天至少集成一次，也就意味着每天可能会发生多次集成。每次集成都通过自动化的构建(包括编译，发布，自动化测试)来验证，从而尽快地发现集成错误。

- 构建物例如Java中的Jar包

Python中的egg



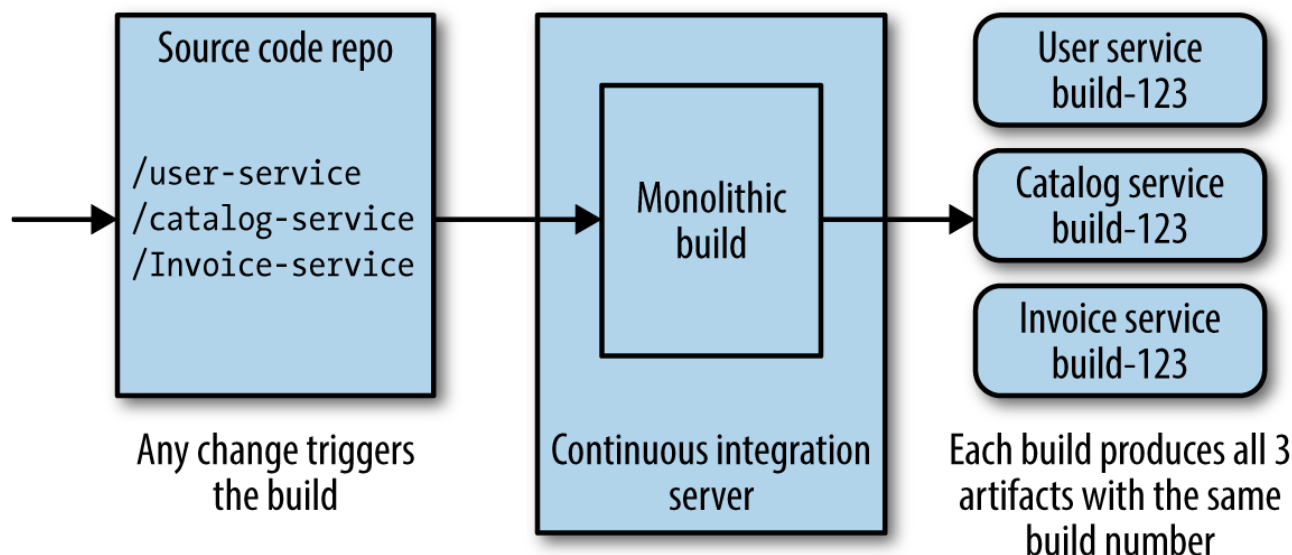
西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

6.1 把CI映射到微服务

- 1 把所有微服务放在同一个代码库中，并且只有一个CI构建

不推荐

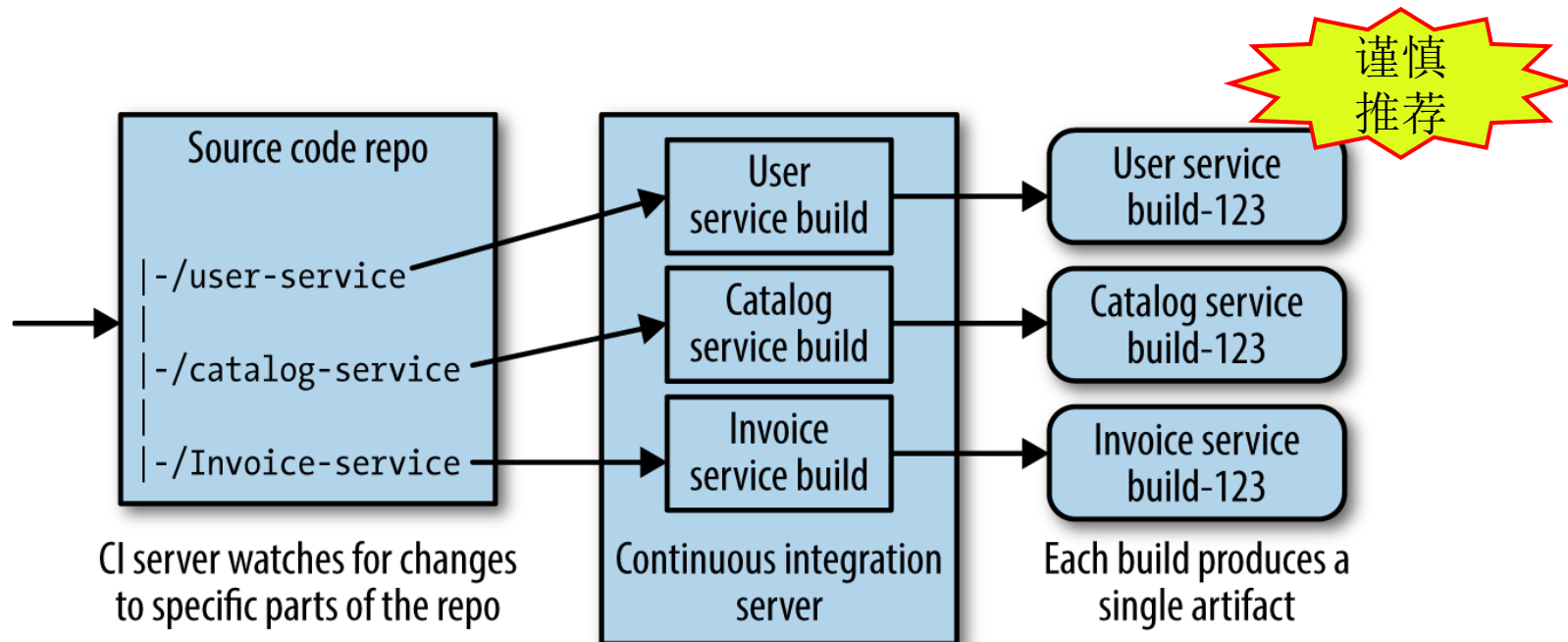


- 优点：简单，只需提交代码，若需要 **同时** 在多个服务上工作，一个提交就可以。适合项目初期单一团队工作
- 缺点：每次代码修改，都需要验证和构建所有服务



6.1 把CI映射到微服务

- 2 将一个代码库的子目录映射到不同的构建中

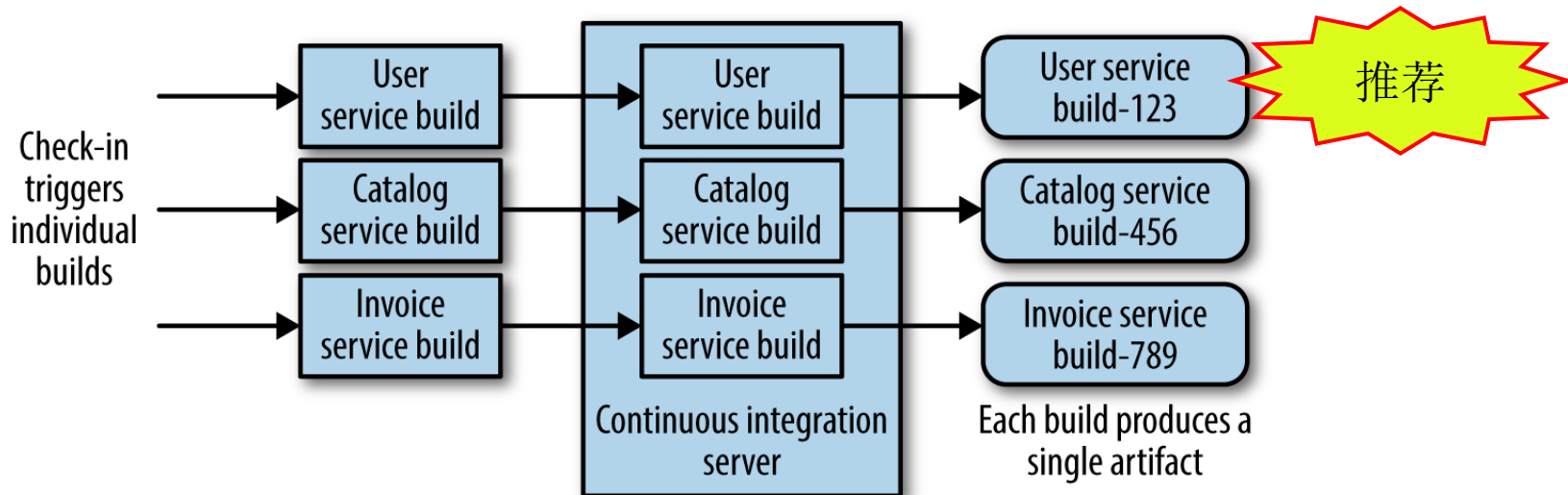


- 优点：若代码目录结构合理，则服务可以分别构建。保持了代码检入/检出流程的简洁
- 缺点：有将多个服务耦合在一起修改的倾向



6.1 把CI映射到微服务

● 3 每个微服务都有一个源代码库和CI构建



- 优点：每个微服务都有自己的代码库，当代码修改时可以只运行相关的构建以及其中的测试，代码库与团队所有权的匹配度更高
- 缺点：跨服务修改比较困难，但比其他两种存在的问题要简单得多



6.2 构建流水线与持续交付

● 构建流水线

○ 将构建过程分阶段完成具有很高价值

- 测试中有些属于运行很快、涉及影响范围小；还有一些较耗时、涉及范围大，它们一起运行弊端很多
- 将构建分成多个阶段，这就是 **构建流水线**

○ 构建流水线能很好地跟踪软件构建进度



● 持续交付（**Continuous Delivery, CD**）

- 是构建流水线的发展，能够检查每次提交是否达到了部署到生产环境的要求，并持续反馈
- 每次提交都可当作候选发布版本



6.2 构建流水线与持续交付

- 每个微服务都有自己独立的CI
- 每个微服务对应一个构建
- 例外情况：
 - 项目刚启动，服务边界不清晰，在识别出稳定的领域边界之前，所有初始服务放在一起
 - 所有服务放入一个单独构建中，有利于减轻跨服务修改所带来的代价
 - 当服务的API稳定后，就可以开始把它们移动到各自的构建中

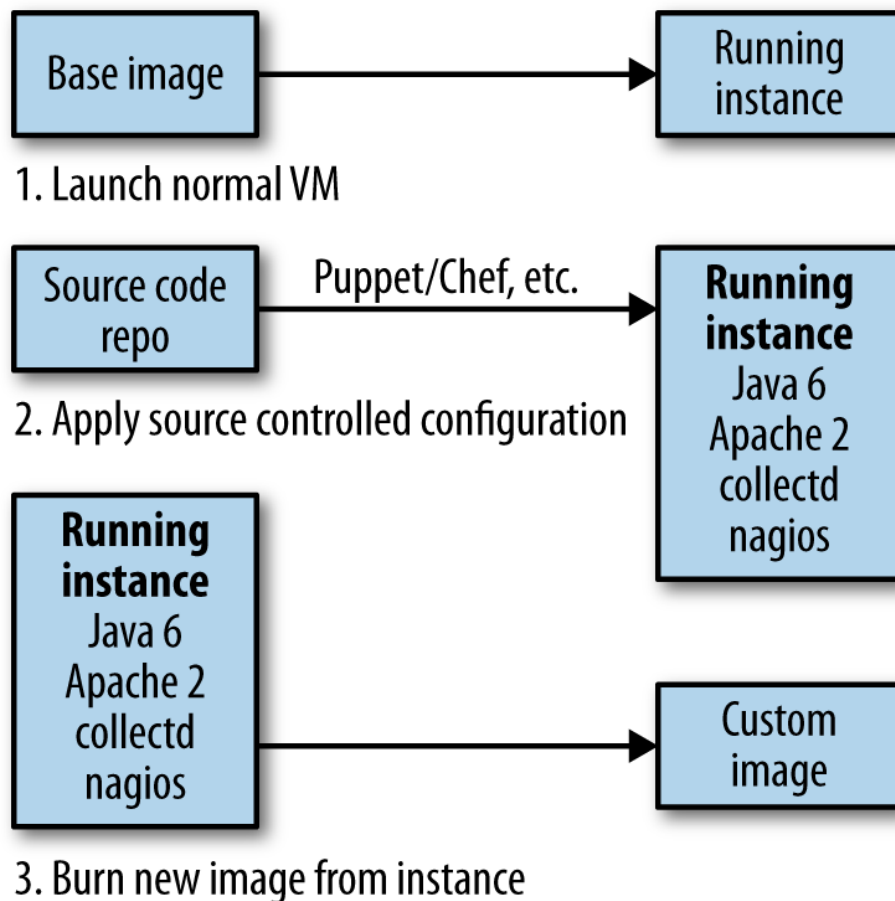


6.3 以镜像作为构建物的一部分

- 为构建提供环境很复杂
 - 服务依赖的平台环境作为构建物的一部分
 - 操作系统作为构建物的一部分
- 将虚拟机镜像作为构建物的一部分
 - 镜像中安装了公共工具
 - 部署软件时，只需根据该镜像创建一个实例，之后在其上安装最新的软件版本即可



6.3 以镜像作为构建物的一部分

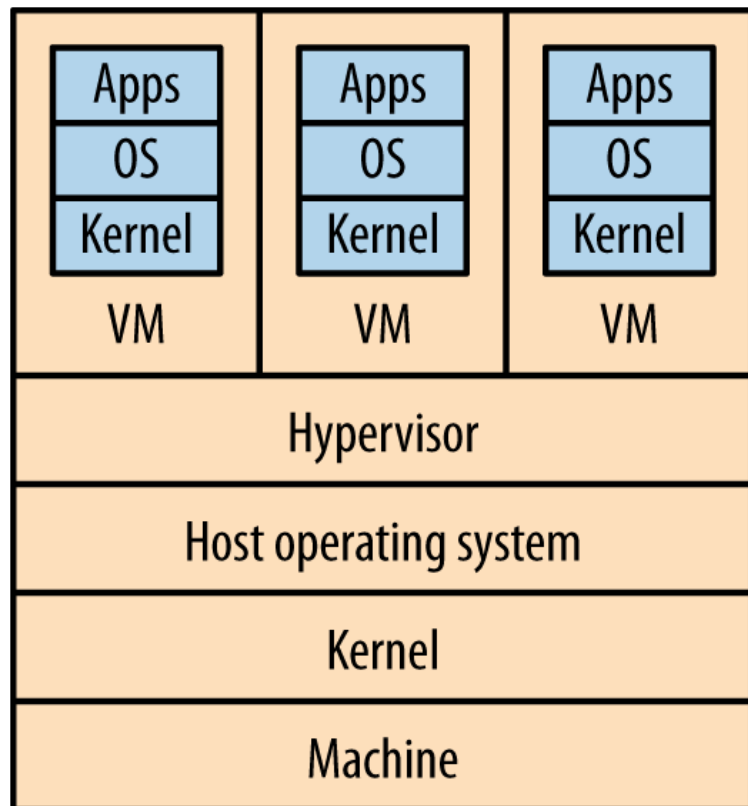


6.4 从物理机到虚拟机

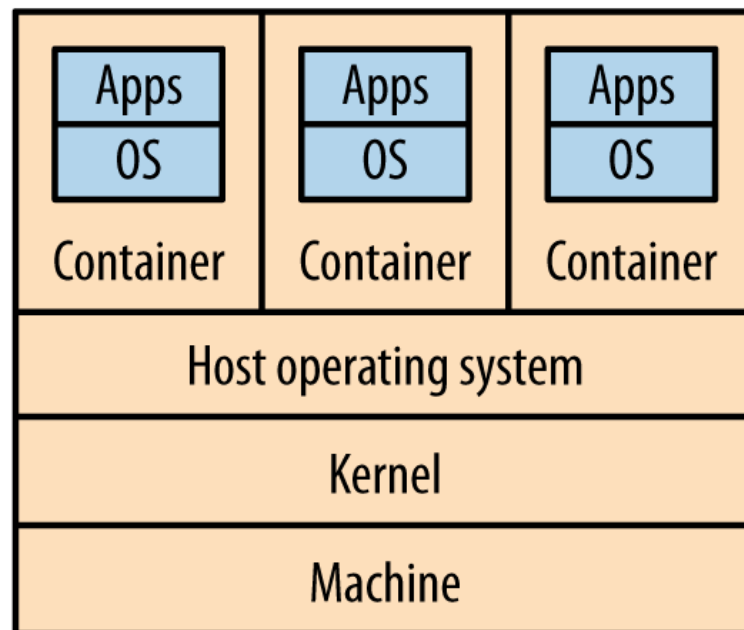
- 虚拟化技术可以大大减少管理主机的开销
 - 把一台物理机分成多台独立的主机，每个服务部署在不同主机上
 - 从标准虚拟化到容器虚拟化
 - 传统虚拟化技术需要额外开销，通过**Hypervisor**来隔离
 - **Linux**容器可以创建一个隔离的进程空间，进而在这个空间中运行其他进程
 - 不需要**Hypervisor**
 - 尽管每个容器可以运行不同的操作系统发行版本，但必须共享相同的内核，如主机运行**Ubuntu**，容器中运行**CentOS**
 - 速度非常快
 - 轻量级，可以运行很多容器数量



6.4 从物理机到虚拟机



Standard virtualizations



Container-based virtualizations (LXC)



6.4 从物理机到虚拟机

